

Deep Q-Network for ATARI

Reinforcement Learning Assignment 1 — BIU 2026

Yonatan Kapach — 318515756

May 2026

Contents

1	Game Description	2
2	Preprocessing	2
3	Network Architecture	2
4	Hyperparameters	3
5	Experiments: What Worked and What Did Not	4
5.1	What Worked	4
5.2	What Did Not Work	4
6	Results	5
6.1	Evaluation Protocol	5
6.2	Learning Curves	5
6.3	Final Scores	6

1 Game Description

The chosen game is **Crazy Climber** (Atari 2600), accessed via the Arcade Learning Environment (ALE) through the Gymnasium interface (`ALE/CrazyClimber-v5`). In Crazy Climber the player controls a man climbing a skyscraper; points are awarded for ascending each floor and for reaching the top of each building. The game emits strictly non-negative rewards, with common per-step bonuses ranging from 100 to 400 points. The key challenge for an RL agent is the long horizon required to reach high floors and accumulate large rewards.

Published benchmark scores for Crazy Climber:

- Random agent: 10,781
- Best linear features agent: 23,411
- DQN paper: 114,103

2 Preprocessing

The preprocessing pipeline exactly reproduces Section 4.1 of Mnih et al. (2013), implemented using standard Gymnasium wrappers with no external RL library logic. We used (`ALE/CrazyClimber-v5`) instead of the required (`ALE/CrazyClimber-NoFrameskip-v4`) simply because the NoFrameskip version was not available in the Gymnasium registry at the time of implementation. The parameters of the environment were set to match the required NoFrameskip-v4 ones.

1. **No-op reset.** At the start of each episode a random number of no-operation actions (uniformly sampled from $[1, 30]$) are taken to introduce stochasticity in the initial state (`noop_max=30`).
2. **Frame skipping.** Each selected action is repeated for 4 consecutive frames; the observation returned is the pixel-wise maximum over the last 2 frames to remove flickering artefacts (`frame_skip=4`).
3. **Grayscale and resize.** Each frame is converted to grayscale and resized to 84×84 pixels using `AtariPreprocessing`.
4. **Frame stacking.** The 4 most recent processed frames are stacked along the channel dimension, yielding a state tensor of shape $(4, 84, 84)$ (`FrameStackObservation`).
5. **Sticky actions disabled.** `repeat_action_probability=0.0` disables sticky actions, matching the NoFrameskip-v4 deterministic-action setting required by the assignment.
6. **Reward transformation.** Applied during training only; raw game scores are used during evaluation so that reported rewards are comparable to published benchmarks (see Section 5).

3 Network Architecture

The Q-network follows the convolutional architecture from the DQN paper. Input observations are stored as `uint8` in the replay buffer and normalised to $[0, 1]$ (divided by 255) before each forward pass.

Table 1: DQN network architecture

Layer	Type	Parameters	Output shape
Input	—	—	$4 \times 84 \times 84$
Conv1	Conv2d + ReLU	32 filters, 8×8 , stride 4	$32 \times 20 \times 20$
Conv2	Conv2d + ReLU	64 filters, 4×4 , stride 2	$64 \times 9 \times 9$
Conv3	Conv2d + ReLU	64 filters, 3×3 , stride 1	$64 \times 7 \times 7$
Flatten	—	—	3,136
FC1	Linear + ReLU	512 units	512
Output	Linear	n_{actions} units	9

Crazy Climber exposes 9 discrete actions. The loss is the Huber loss (smooth- ℓ_1) applied to temporal-difference errors computed against a periodically frozen *target network*:

$$y_i = \tilde{r}_i + \gamma \max_{a'} Q_{\theta^-}(s'_i, a'), \quad \mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^B \ell_\delta(y_i - Q_\theta(s_i, a_i)), \quad (1)$$

where θ^- denotes the target-network parameters, $\gamma = 0.99$, and ℓ_δ is the Huber loss with $\delta = 1$.

4 Hyperparameters

Table 2: Hyperparameters used for the three final submitted runs

Hyperparameter	Value
Optimizer	Adam
Learning rate	1×10^{-4}
Discount factor γ	0.99
Batch size	32
Replay buffer capacity	150,000 transitions
Minimum replay size (warm-up)	20,000 transitions
Network update frequency	Every 4 environment steps
Target network sync frequency	Every 10,000 environment steps
ε -start (training)	1.0
ε -end (training)	0.1
ε decay schedule	Linear over 1,000,000 steps
ε during evaluation	0.05
Frame skip	4
Frame stack (n)	4
Reward mode	Clip $\rightarrow \{-1, 0, +1\}$
Total training steps	2,000,000 (Run 1); 1,900,000 (Runs 2–3)
Evaluation frequency	Every 10,000 training steps
Evaluation episodes per checkpoint	30

Replay buffer capacity. The DQN paper uses a buffer of 1,000,000 transitions. To remain within the host-RAM limits of a Google Colab GPU instance the capacity was reduced to 150,000. The agent still learns effectively at this size; transitions age out faster but the diversity requirement is met.

5 Experiments: What Worked and What Did Not

5.1 What Worked

Adam optimizer. Switching from RMSProp (the paper’s default) to Adam was the single most impactful change. Adam’s adaptive first- and second-moment estimates produce faster initial learning and more stable convergence on Crazy Climber. Adam produced measurable reward improvements within the first 50,000 steps.

Reward clipping to $\{-1, 0, +1\}$. Clipping rewards normalises the magnitude of TD targets and prevents Q-value divergence, a critical stabilisation technique from the original paper. For Crazy Climber, which emits only non-negative rewards, reward clipping simplifies to:

$$\tilde{r} = \begin{cases} 1 & \text{if } r > 0, \\ 0 & \text{if } r = 0. \end{cases} \quad (2)$$

This provides a uniform, strong gradient signal for every positive transition, enabling the agent to focus on maximising the visit frequency of rewarding states rather than on reward magnitude differentiation.

30 evaluation episodes per checkpoint. Using 30 episodes per checkpoint reduces the variance of the mean-reward estimate and produces a more stable selection signal (but still noisy) compared to using only a few single episodes.

5.2 What Did Not Work

RMSProp optimizer. The paper specifies RMSProp with $\alpha = 0.95$, $\varepsilon = 0.01$, and $\text{lr} = 2.5 \times 10^{-4}$. On Crazy Climber this configuration showed no learning within the first several hundred thousand steps: mean evaluation rewards remained at or below the random-agent baseline ($\approx 10,781$). The run was terminated early and Adam was adopted in all subsequent experiments.

Reward normalization. As an alternative to hard clipping, a soft *reward normalization* scheme was tested:

$$\tilde{r} = \min\left(\frac{r}{r_{\max}}, 1\right) = \text{clip}\left(\frac{r}{400}, 0, 1\right), \quad (3)$$

where $r_{\max} = 400$ is the maximum dense per-step reward in Crazy Climber (the climbing bonus on Building 4).

The intent was to preserve ordinal reward structure: under (3), $r = 100 \Rightarrow \tilde{r} = 0.25$, $r = 200 \Rightarrow \tilde{r} = 0.50$, and $r = 400 \Rightarrow \tilde{r} = 1.0$. In contrast, hard clipping collapses all positive rewards to $\tilde{r} = 1$, discarding magnitude information but providing a uniform gradient signal.

Normalization proved **less successful** for the following reasons:

- **Weaker early signal.** The Bellman target is $y = \tilde{r} + \gamma \max_{a'} Q(s', a')$. Under normalization, the most common small rewards ($r \approx 100$) contribute only $\tilde{r} = 0.25$ to the target, compared to $\tilde{r} = 1$ under clipping — a $4\times$ reduction in the immediate reward component. This significantly slows early learning when the Q-values are still near zero and the bootstrapped term is uninformative.
- **Non-uniform gradient magnitudes.** Different reward magnitudes produce different TD-error scales, requiring the critic to simultaneously handle a fine-grained reward scale and the bootstrapping signal. This introduces additional instability compared to the simpler binary signal from clipping.

- **Empirical outcome.** The normalization run peaked at a mean evaluation reward of approximately 66,700 (with 10 evaluation episodes per checkpoint), compared to peaks of $\sim 92,000$ – $96,000$ under reward clipping with 30 evaluation episodes.

Hard clipping was retained for all three final reported runs.

6 Results

6.1 Evaluation Protocol

- Evaluation frequency: every 10,000 environment steps.
- Episodes per evaluation checkpoint: **30**.
- Evaluation policy: ε -greedy with $\varepsilon = 0.05$.
- Rewards recorded: raw (unclipped) game scores.

Step selection methodology. To avoid overfitting the reported score to any individual run, a single evaluation step was selected *using only Run 1* and then applied unchangeably to all three runs:

1. Run 1 was trained for 2,000,000 steps (199 evaluation checkpoints).
2. The checkpoint with the highest mean evaluation reward in Run 1 was identified as step 1,900,000. (There was actually a slightly higher reward in a much earlier step, but I interpreted that as an outlier and chose the more stable later peak)
3. Runs 2 and 3 were trained for exactly 1,900,000 steps (the same budget) using different random seeds.
4. The final score for each run is the mean evaluation reward at step 1,900,000, and the aggregate result is the average of those three values.

This procedure ensures that the reported step is not cherry-picked across runs, keeping the evaluation protocol unbiased.

6.2 Learning Curves

Figures 1–3 show the mean evaluation reward vs. training steps for each of the three independent final runs. Each point corresponds to one evaluation checkpoint (30 episodes, $\varepsilon = 0.05$).

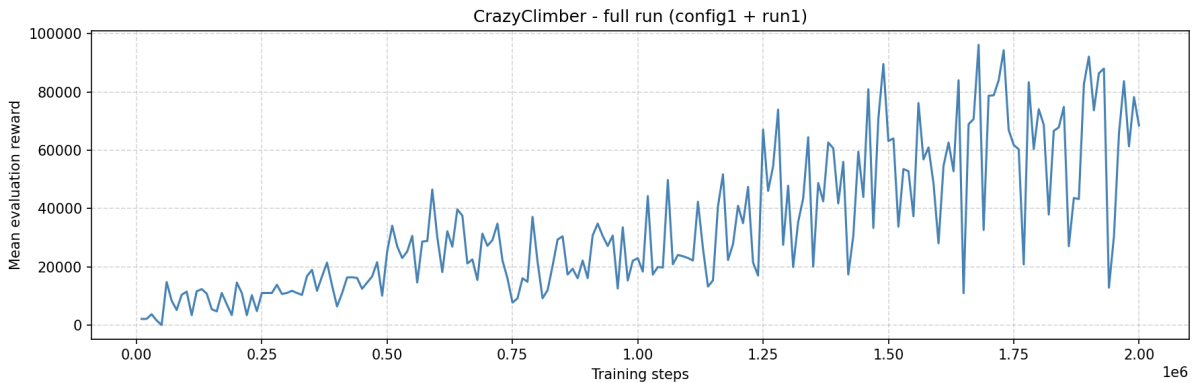


Figure 1: Learning curve — Run 1 (full 2,000,000 steps, used for step selection). Mean evaluation reward at step 1,900,000: 92,160.

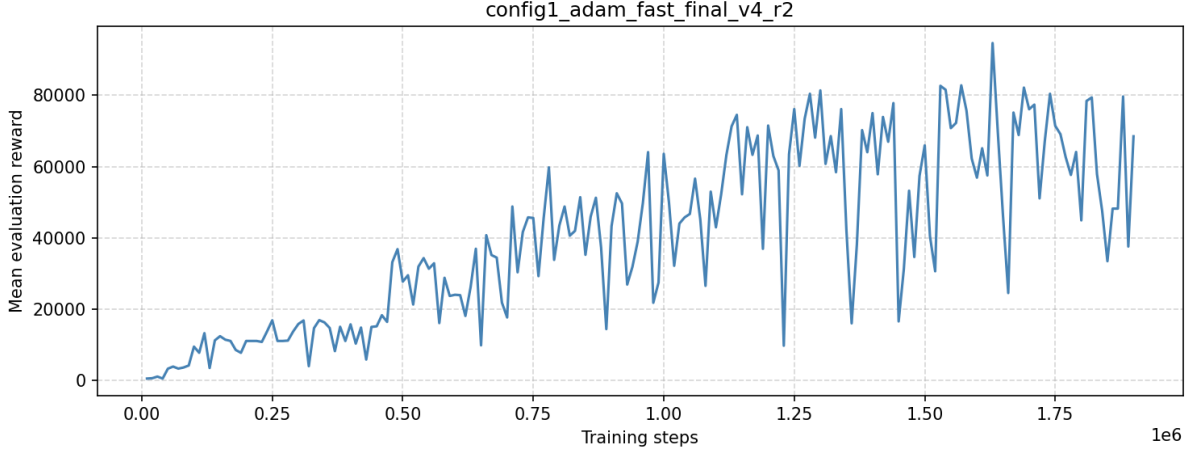


Figure 2: Learning curve — Run 2 (1,900,000 steps). Mean evaluation reward at step 1,900,000: 68,497.

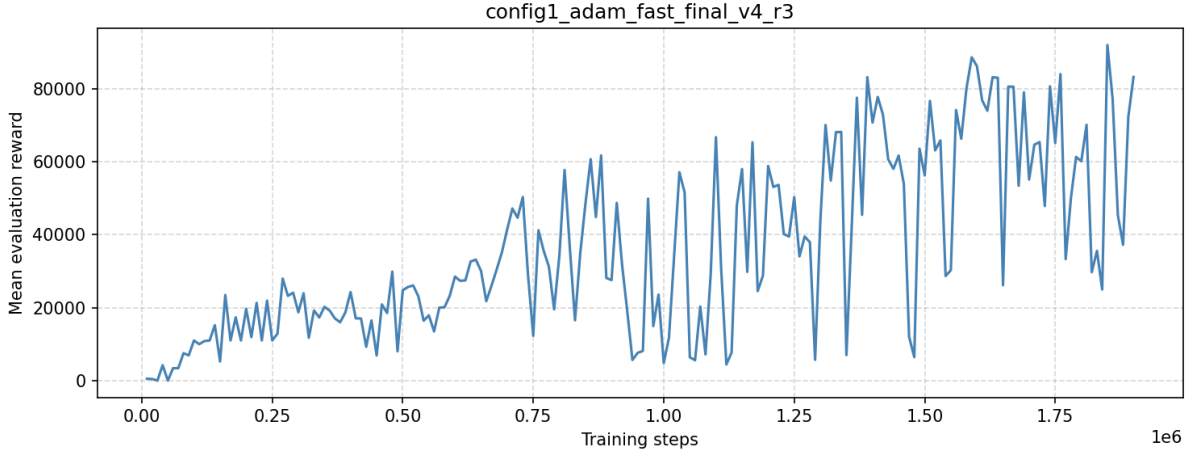


Figure 3: Learning curve — Run 3 (1,900,000 steps). Mean evaluation reward at step 1,900,000: 83,243.

6.3 Final Scores

The reported score for each run is the mean evaluation reward at the pre-selected step 1,900,000 (30 episodes, $\varepsilon = 0.05$).

Table 3: Final results — three independent runs at step 1,900,000

Run	Seed	Mean eval reward at step 1,900,000
Run 1	42	92,160
Run 2	100	68,497
Run 3	467	83,243
Average	—	81,300

For reference: random agent = 10,781; best linear agent = 23,411; DQN paper = 114,103. The achieved average of **81,300** far exceeds the linear agent target and reaches $\approx 71\%$ of the DQN

paper score in 1,900,000 training steps. The learning curve seem to continue trending upwards at the end of training, suggesting that further training could yield even higher scores. I tend to believe that with better computational resources and more training steps, the agent could reach or even surpass the DQN paper score.